Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра интеллектуальных информационных технологий

Лабораторная работа № 6
По дисциплине: «Современные платформы программирования»
За 6 семестр

Выполнил:
Студент 3 курса
Группы ПО-12
Гомберг Д.П.
Проверил:
Кулик А.Д.

Брест 2026

Цель: освоить приемы тестирования кода на примере использования пакета pytest

Вариант 3

Задание 1

Написание тестов для мини-библиотеки покупок (shopping.py)

1. Создайте файл test_cart.py. Реализуйте следующие тесты:

• Проверка добавления товара: после add_item("Apple", 10.0) в корзине должен быть один элемент.

• Проверка выброса ошибки при отрицательной цене.

• Проверка вычисления общей стоимости (total()).

2. Протестируйте метод apply_discount с разными значениями скидки:

• 0% - цена остаётся прежней

• 50% - цена уменьшается вдвое

• 100% - цена становится ноль

• < 0% и > 100% - должно выбрасываться исключение

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы No1. Проверьте тривиальные и граничные случае, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.3) Напишите метод String keep(String str, String pattern) который оставляет

в первой строке все символы, которые присутствуют во второй.

Спецификация метода:

keep (None , None) = TypeError

keep (None, *) = None

keep ("", *) = ""

keep (* , None) = ""

keep (* , "") = ""

keep (" hello ", "hl") = " hll "

keep (" hello ", "le") = " ell "

Код программы:

```
import pytest
import requests
from unittest.mock import patch, MagicMock


class Cart:
    def __init__(self):
        self.items = []

    def add_item(self, name: str, price: float):
        if price < 0:
            raise ValueError("Price cannot be negative")
        self.items.append({"name": name, "price": price})

    def total(self) -> float:
        return sum(item["price"] for item in self.items)

    def apply_discount(self, discount_percent: float):
        if discount_percent < 0 or discount_percent > 100:
            raise ValueError("Discount must be between 0 and 100")
        for item in self.items:
            item["price"] = item["price"] * (1 - discount_percent / 100)

    def apply_coupon(self, coupon_code: str):
        coupons = {"SAVE10": 10, "HALF": 50}
        if coupon_code in coupons:
            self.apply_discount(coupons[coupon_code])
        else:
            raise ValueError("Invalid coupon")


def log_purchase(item):
    requests.post("https://example.com/log", json=item)


# ============ TESTS FOR TASK 1 ============


class TestCart:
    def test_add_item(self, empty_cart):
        empty_cart.add_item("Apple", 10.0)
```

```
        assert len(empty_cart.items) == 1

    def test_add_item_negative_price_raises_error(self, empty_cart):
        with pytest.raises(ValueError, match="Price cannot be negative"):
            empty_cart.add_item("Apple", -10.0)

    def test_total(self, empty_cart):
        empty_cart.add_item("Apple", 10.0)
        empty_cart.add_item("Banana", 5.0)
        assert empty_cart.total() == 15.0

    @pytest.mark.parametrize(
        "discount,expected_price",
        [
            (0, 100.0),
            (50, 50.0),
            (100, 0.0),
        ],
    )
    def test_apply_discount(self, empty_cart, discount, expected_price):
        empty_cart.add_item("Item", 100.0)
        empty_cart.apply_discount(discount)
        assert empty_cart.items[0]["price"] == expected_price

    @pytest.mark.parametrize("invalid_discount", [-1, 101])
    def test_apply_discount_invalid_raises_error(self, empty_cart, invalid_discount):
        empty_cart.add_item("Item", 100.0)
        with pytest.raises(ValueError, match="Discount must be between 0 and 100"):
            empty_cart.apply_discount(invalid_discount)


class TestLogPurchase:
    @patch("task1.requests.post")
    def test_log_purchase_calls_post_with_correct_data(self, mock_post):
        item = {"name": "Apple", "price": 10.0}
        log_purchase(item)
        mock_post.assert_called_once_with("https://example.com/log", json=item)


class TestApplyCoupon:
    @pytest.mark.parametrize(
        "coupon,discount",
        [
            ("SAVE10", 10),
            ("HALF", 50),
        ],
    )
    def test_apply_coupon_valid(self, empty_cart, coupon, discount):
        empty_cart.add_item("Item", 100.0)
```

```
        empty_cart.apply_coupon(coupon)
        assert empty_cart.items[0]["price"] == 100.0 * (1 - discount / 100)

    def test_apply_coupon_invalid_raises_error(self, empty_cart):
        empty_cart.add_item("Item", 100.0)
        with pytest.raises(ValueError, match="Invalid coupon"):
            empty_cart.apply_coupon("INVALID")

    def test_apply_coupon_with_monkeypatch_coupons_dict(self, empty_cart, monkeypatch):
        original_apply_coupon = Cart.apply_coupon

        def patched_apply_coupon(self, coupon_code):
            test_coupons = {"TEST": 25}
            if coupon_code in test_coupons:
                self.apply_discount(test_coupons[coupon_code])
            else:
                raise ValueError("Invalid coupon")

        monkeypatch.setattr(Cart, "apply_coupon", patched_apply_coupon)
        empty_cart.add_item("Item", 100.0)
        empty_cart.apply_coupon("TEST")
        assert empty_cart.items[0]["price"] == 75.0


@pytest.fixture
def empty_cart():
    return Cart()


if __name__ == "__main__":
    pytest.main([__file__, "-v"])


import pytest
import sys
import os

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "..", "Lab1", "src"))

from Test1 import find_mode
from Test2 import str_str


class TestFindMode:
    def test_empty_sequence_returns_none(self):
        assert find_mode([]) is None

    def test_no_mode_all_elements_same_frequency(self):
        assert find_mode([1, 2, 3]) is None
```

```python
    def test_single_mode(self):
        assert find_mode([1, 2, 2, 3]) == [2]

    def test_multiple_modes(self):
        result = find_mode([1, 1, 2, 2, 3])
        assert set(result) == {1, 2}

    def test_single_element(self):
        assert find_mode([5]) is None

    def test_all_same_elements(self):
        assert find_mode([7, 7, 7]) is None

    def test_negative_numbers(self):
        result = find_mode([-1, -1, -2, -2, -3])
        assert set(result) == {-1, -2}

    def test_strings(self):
        result = find_mode(["a", "b", "a", "c", "b"])
        assert set(result) == {"a", "b"}


class TestStrStr:
    def test_empty_haystack_returns_negative(self):
        assert str_str("", "abc") == -1

    def test_empty_needle_returns_zero(self):
        assert str_str("abc", "") == 0

    def test_both_empty_returns_zero(self):
        assert str_str("", "") == 0

    def test_needle_not_found(self):
        assert str_str("hello", "world") == -1

    def test_needle_found_at_beginning(self):
        assert str_str("hello", "he") == 0

    def test_needle_found_at_end(self):
        assert str_str("hello", "lo") == 3

    def test_needle_found_in_middle(self):
        assert str_str("ffsadbutsad", "sad") == 2

    def test_needle_longer_than_haystack(self):
        assert str_str("abc", "abcd") == -1

    def test_exact_match(self):
        assert str_str("abc", "abc") == 0
```

```
    def test_overlapping_needle(self):
        assert str_str("aaa", "aa") == 0


if __name__ == "__main__":
    pytest.main([__file__, "-v"])


import pytest


def keep(s: str, pattern: str) -> str:
 if s is None and pattern is None:
 raise TypeError("Both arguments cannot be None")

 if s is None:
 return None
 if pattern is None:
 return ""
 if s == "" or pattern == "":
 return ""

 result = ""
 for char in s:
 if char in pattern:
 result += char

 return result


class TestKeep:
 def test_both_none_raises_type_error(self):
 with pytest.raises(TypeError):
 keep(None, None)

 def test_first_none_returns_none(self):
 assert keep(None, "abc") is None

 def test_first_empty_returns_empty(self):
 assert keep("", "abc") == ""

 def test_second_none_returns_empty(self):
 assert keep("abc", None) == ""

 def test_second_empty_returns_empty(self):
 assert keep("abc", "") == ""

 def test_keep_hl(self):
 assert keep(" hello ", "hl") == "hll"
```

```
def test_keep_le(self):
assert keep(" hello ", "le") == "ell"

    def test_keep_no_match(self):
        assert keep("abc", "xyz") == ""

    def test_keep_all_match(self):
        assert keep("aaa", "a") == "aaa"

    def test_pattern_longer_than_string(self):
        assert keep("ab", "abcd") == "ab"

    def test_keep_with_duplicates_in_pattern(self):
        result = keep("hello", "ll")
        assert result == "ll"


if __name__ == "__main__":
    pytest.main([__file__, "-v"])
```

Результат выполнения программы:

```
spp_po12/reports/Gomberg_D_P/Lab6/src/task1.py::TestCart::test_add_item PASSED [  7%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task1.py::TestCart::test_add_item_negative_price_raises_error PASSED [ 15%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task1.py::TestCart::test_total PASSED [ 23%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task1.py::TestCart::test_apply_discount[0-100.0] PASSED [ 30%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task1.py::TestCart::test_apply_discount[50-50.0] PASSED [ 38%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task1.py::TestCart::test_apply_discount[100-0.0] PASSED [ 46%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task1.py::TestCart::test_apply_discount_invalid_raises_error[-1] PASSED [ 53%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task1.py::TestCart::test_apply_discount_invalid_raises_error[101] PASSED [ 61%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task1.py::TestLogPurchase::test_log_purchase_calls_post_with_correct_data PASSED [ 69%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task1.py::TestApplyCoupon::test_apply_coupon_valid[SAVE10-10] PASSED [ 76%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task1.py::TestApplyCoupon::test_apply_coupon_valid[HALF-50] PASSED [ 84%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task1.py::TestApplyCoupon::test_apply_coupon_invalid_raises_error PASSED [ 92%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task1.py::TestApplyCoupon::test_apply_coupon_with_monkeypatch_coupons_dict PASSED [100%]
```

```
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestFindMode::test_empty_sequence_returns_none PASSED [  5%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestFindMode::test_no_mode_all_elements_same_frequency PASSED [ 11%
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestFindMode::test_single_mode PASSED [ 16%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestFindMode::test_multiple_modes PASSED [ 22%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestFindMode::test_single_element PASSED [ 27%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestFindMode::test_all_same_elements PASSED [ 33%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestFindMode::test_negative_numbers PASSED [ 38%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestFindMode::test_strings PASSED [ 44%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestStrStr::test_empty_haystack_returns_negative PASSED [ 50%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestStrStr::test_empty_needle_returns_zero PASSED [ 55%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestStrStr::test_both_empty_returns_zero PASSED [ 61%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestStrStr::test_needle_not_found PASSED [ 66%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestStrStr::test_needle_found_at_beginning PASSED [ 72%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestStrStr::test_needle_found_at_end PASSED [ 77%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestStrStr::test_needle_found_in_middle PASSED [ 83%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::TestStrStr::test_needle_longer_than_haystack PASSED [ 88%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::Tests                                    D [ 94%]
spp_po12/reports/Gomberg_D_P/Lab6/src/task2.py::Test                                     PASSED [100%]
```

Динамики

Вывод: освоили приемы тестирования кода на примере использования пакета pytest